

2-7 设 $P(x)=a_0+a_1x+\cdots+a_dx^d$ 是一个 d 次多项式。假设已有一算法能在 $O(i)$ 时间内计算一个 i 次多项式与一个一次多项式的乘积，以及一个算法能在 $O(i\log i)$ 时间内计算两个 i 次多项式的乘积。对于任意给定的 d 个整数 $n_1, n_2, \dots, n_d$ ，用分治法设计一个有效算法，计算出满足 $P(n_1)=P(n_2)=\cdots=P(n_d)=0$ 且最高次项系数为 1 的 d 次多项式 $P(x)$ ，并分析算法的效率。

1. 将 d 个点分成两组：
 - 左半部分： $n_1, n_2, \dots, n_{\lfloor d/2 \rfloor}$
 - 右半部分： $n_{\lfloor d/2 \rfloor + 1}, \dots, n_d$
2. 递归地为每一部分构造多项式：
 - $L(x)$ ：左半部分的多项式
 - $R(x)$ ：右半部分的多项式
3. 最终多项式 $P(x) = L(x) \times R(x)$

时间复杂度分析：设 $T(d)$ 为处理 d 个点的时间复杂度：

$$T(d) = 2T(d/2) + O(d \log d) \quad d > 1$$

$$T(d) = O(1) \quad d = 1$$

根据主定理，最终时间复杂度为 $O(d \log^2 d)$

2-15 设有 n 个运动员要进行网球循环赛。设计一个满足以下要求的比赛日程表：

- (1) 每个选手必须与其他 $n-1$ 个选手各赛一次；
- (2) 每个选手一天只能赛一次；
- (3) 当 n 是偶数时，循环赛进行 $n-1$ 天。当 n 是奇数时，循环赛进行 n 天。

```
vector<vector<int>>> a; // 存储比赛日程表
```

```
// 判断是否为奇数
```

```
bool odd(int n) {
    return n & 1;
}
```

```
// 复制奇数情况下的日程表
```

```
void copyodd(int n) {
    int m = n/2;
    // 第一步：复制上半部分到下半部分
    for(int i = 1; i <= m; i++) {
        a[i][m+1] = a[i][1];
        a[m+i][m+1] = a[m+i][1];
    }
}
```

```
// 第二步：处理剩余部分
```

```
for(int i = 1; i <= m; i++) {
    for(int j = 1; j <= m; j++) {
        if(a[i][j] > m) {
```

```

        a[i][j+m] = a[i][j];
        a[m+i][j] = a[i][j] % n;
    } else {
        a[m+i][j] = a[i][j] + m;
    }
}

}

// 第三步: 填充最后一列
for(int j = 2; j <= m; j++) {
    a[1][m+j] = a[1+j-1][m+1];
    a[1+j-1][m+j] = 1;
}
}

// 复制偶数情况下的日程表
void copy(int n) {
    int m = n/2;
    for(int i = 1; i <= m; i++) {
        for(int j = 1; j <= n-1; j++) {
            a[m+i][j] = a[i][j] + m;
        }
    }
}

// 根据 n 的奇偶性选择不同的复制方法
void makecopy(int n) {
    if(n/2 > 1 && odd(n/2)) {
        copyodd(n);
    } else {
        copy(n);
    }
}

// 递归生成日程表
void tournament(int n) {
    if(n == 1) {
        a[1][1] = 1;
        return;
    }

    if(odd(n)) {
        tournament(n+1);
        return;
    }

    tournament(n/2);

```

```

    makecopy(n);
}

```

时间复杂度分析:

每次递归将问题规模减半 (忽略奇数情况的临时+1)

每层的复制操作需要 $O(n^2)$ 时间

递归深度为 $\log n$

因此总时间复杂度为:

$T(n)=O(n^2\log n)$

2-6 排列的字典序问题。

问题描述: n 个元素 $\{1, 2, \dots, n\}$ 有 $n!$ 个不同的排列。将这 $n!$ 个排列按字典序排列, 并编号为 $0, 1, \dots, n!-1$ 。每个排列的编号为其字典序值。例如, 当 $n=3$ 时, 6 个不同排列的字典序值如下:

字典序值	0	1	2	3	4	5
排列	123	132	213	231	312	321

算法设计: 给定 n 及 n 个元素 $\{1, 2, \dots, n\}$ 的一个排列, 计算出这个排列的字典序值, 以及按字典序排列的下一个排列。

数据输入: 由文件 input.txt 提供输入数据。文件的第 1 行是元素个数 n 。接下来的 1 行

042

是 n 个元素 $\{1, 2, \dots, n\}$ 的一个排列。

结果输出: 将计算出的排列的字典序值和按字典序排列的下一个排列输出到文件 output.txt。文件的第 1 行是字典序值, 第 2 行是按字典序排列的下一个排列。

输入文件示例
input.txt
8
2 6 4 5 8 1 7 3

输出文件示例
output.txt
8227
2 6 4 5 8 3 1 7

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <fstream>
using namespace std;

class Solution {
public:
    // 计算排列的字典序值
    int getPermutationValue(const vector<int>& perm) {
        int n = perm.size();
        int value = 0;

```

```

vector<bool> used(n + 1, false);

for(int i = 0; i < n; i++) {
    int smaller = 0;
    // 统计当前位置之前有多少个小于当前数的未使用数字
    for(int j = 1; j < perm[i]; j++) {
        if(!used[j]) smaller++;
    }
    // 计算贡献值
    value += smaller * factorial(n - 1 - i);
    used[perm[i]] = true;
}

return value;
}

// 根据字典序值获取下一个排列
vector<int> getNextPermutation(const vector<int>& perm) {
    vector<int> next = perm;
    int n = next.size();

    // 从右向左找到第一个降序的位置
    int i = n - 2;
    while(i >= 0 && next[i] >= next[i + 1]) {
        i--;
    }

    if(i >= 0) {
        // 从右向左找到第一个大于next[i]的数
        int j = n - 1;
        while(j > i && next[j] <= next[i]) {
            j--;
        }

        // 交换这两个数
        swap(next[i], next[j]);
    }

    // 将i之后的数字反转
    reverse(next.begin() + i + 1, next.end());

    return next;
}

```

```

private:
    // 计算阶乘
    int factorial(int n) {
        int result = 1;
        for(int i = 2; i <= n; i++) {
            result *= i;
        }
        return result;
    }
};

int main() {
    ifstream fin("input.txt");
    ofstream fout("output.txt");

    int n;
    fin >> n;

    vector<int> perm(n);
    for(int i = 0; i < n; i++) {
        fin >> perm[i];
    }

    Solution sol;
    int value = sol.getPermutationValue(perm);
    vector<int> next = sol.getNextPermutation(perm);

    // 输出结果
    fout << value << endl;
    for(int i = 0; i < n; i++) {
        fout << next[i] << (i == n-1 ? "" : " ");
    }

    fin.close();
    fout.close();

    return 0;
}

```

2-9 双色 Hanoi 塔问题。

问题描述：设 A 、 B 、 C 是 3 个塔座。开始时，在塔座 A 上有一叠共 n 个圆盘，这些圆盘自下而上，由大到小地叠放在一起。各圆盘从小到大编号为 $1, 2, \dots, n$ ，奇数号圆盘着红色，偶数号圆盘着蓝色，如图 2-19 所示。现要求将塔座 A 上的这一叠圆盘移到塔座 B 上，并仍按同样顺序叠置。在移动圆盘时应遵守以下移动规则：

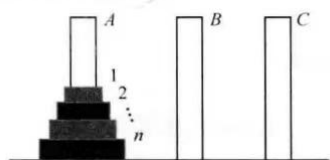


图 2-19 双色 Hanoi 塔

- 规则 I：每次只能移动 1 个圆盘；
- 规则 II：任何时刻都不允许将较大的圆盘压在较小的圆盘之上；
- 规则 III：任何时刻都不允许将同色圆盘叠放在一起；
- 规则 IV：在满足移动规则 I ~ III 的前提下，可将圆盘移至 A 、 B 、 C 中任一塔座上。

试设计一个算法，用最少的移动次数将塔座 A 上的 n 个圆盘移到塔座 B 上，并仍按同样顺序叠置。

算法设计：对于给定的正整数 n ，计算最优移动方案。

数据输入：由文件 input.txt 给出输入数据。第 1 行是给定的正整数 n 。

结果输出：将计算出的最优移动方案输出到文件 output.txt。文件的每行由一个正整数 k 和 2 个字符 c_1 和 c_2 组成，表示将第 k 个圆盘从塔座 c_1 移到塔座 c_2 上。

输入文件示例

input.txt

3

输出文件示例

output.txt

1 A B

2 A C

1 B C

3 A B

1 C A

2 C B

1 A B

使用数学归纳法证明基础汉诺塔算法不会违反双色汉诺塔规则

归纳假设：当圆盘个数小于 n 时， $\text{hanoi}(n, A, B, C)$ 不违反规则 III，且在移动过程中：

- 目的塔 B 上最低圆盘的编号与 n 具有相同奇偶性
- 辅助塔 C 上最低圆盘的编号与 n 具有不同奇偶性

归纳基础：对于 $n=1$ ，只有一个红色圆盘，显然不会违反规则。

- B 塔上最低圆盘编号为 1，与 $n=1$ 具有相同奇偶性（都是奇数）
- C 塔上没有圆盘，满足条件

对于 $n=2$ ，有一个红色圆盘(1)和一个蓝色圆盘(2)。标准汉诺塔算法的步骤是：

- 将圆盘 1 从 A 移到 C
- 将圆盘 2 从 A 移到 B
- 将圆盘 1 从 C 移到 B

在最终状态：

- B 塔上最低圆盘编号为 2，与 $n=2$ 具有相同奇偶性（都是偶数）
- C 塔上没有圆盘，满足条件

在整个过程中，没有同色圆盘叠放在一起。归纳步骤：当圆盘个数为 n 时，标准汉诺塔算法 $\text{hanoi}(n, A, B, C)$ 由以下 3 个步骤完成：

- $\text{hanoi}(n-1, A, C, B)$
- $\text{move}(A, B)$
- $\text{hanoi}(n-1, C, B, A)$

分析步骤(1)：根据归纳假设，步骤(1)不违反规则 III，且在移动过程中：

- C 塔上最低圆盘的编号与 $n-1$ 具有相同奇偶性

- B 塔上最低圆盘的编号与 $n-1$ 具有不同奇偶性

因此：

- C 塔上最低圆盘的编号与 n 具有不同奇偶性（因为 n 和 $n-1$ 奇偶性不同）
- B 塔上最低圆盘的编号与 n 具有相同奇偶性（因为 n 和 $n-1$ 奇偶性不同）

分析步骤(2)：在步骤(2)中，我们将圆盘 n 从 A 移到 B。

- 圆盘 n 的奇偶性与 n 相同
- B 塔上已有圆盘的最低编号与 n 具有相同奇偶性
- 因此，圆盘 n 与 B 塔上的圆盘不会形成同色相邻

分析步骤(3)：根据归纳假设，步骤(3)不违反规则 III，且在移动过程中：

- B 塔上倒数第 2 个圆盘的编号与 $n-1$ 具有相同奇偶性
- A 塔上最低圆盘的编号与 $n-1$ 具有不同奇偶性

因此：

- B 塔上倒数第 2 个圆盘的编号与 n 具有不同奇偶性（因为 n 和 $n-1$ 奇偶性不同）
- A 塔上最低圆盘的编号与 n 具有相同奇偶性（因为 n 和 $n-1$ 奇偶性不同）

这意味着在 B 塔上，圆盘 n （最低的圆盘）与其上方的圆盘具有不同的奇偶性，因此不会违反规则 III。

最终状态：

- B 塔上最低圆盘的编号为 n ，与 n 具有相同奇偶性
- C 塔上最低圆盘的编号与 n 具有不同奇偶性

通过数学归纳法，证明了标准汉诺塔算法可以直接用于解决双色汉诺塔问题，不会违反规则 III。因此，可以直接使用标准汉诺塔算法来解决双色汉诺塔问题

```
#include <iostream>
#include <fstream>
#include <vector>
using namespace std;

struct Move {
    int disk;
    char from;
    char to;
};

class BicolorHanoi {
private:
    vector<Move> moves;

    // 记录移动
    void recordMove(int disk, char from, char to) {
        moves.push_back({disk, from, to});
    }

    // 标准汉诺塔算法：将 n 个圆盘从 from 移动到 to，使用 aux 作为辅助塔
    void hanoi(int n, char from, char to, char aux) {
        if (n == 0) return;
```

```

        hanoi(n-1, from, aux, to);
        recordMove(n, from, to);
        hanoi(n-1, aux, to, from);
    }

public:
    vector<Move> solve(int n) {
        moves.clear();
        hanoi(n, 'A', 'B', 'C');
        return moves;
    }
};

int main() {
    ifstream fin("input.txt");
    ofstream fout("output.txt");

    int n;
    fin >> n;

    BicolorHanoi solver;
    vector<Move> solution = solver.solve(n);

    for (const Move& move : solution) {
        fout << move.disk << " " << move.from << " " << move.to << endl;
    }

    fin.close();
    fout.close();

    return 0;
}

```